

Problem: Master thesis's "iterative mask-fill" inference is not BEiT-3's intended decoding path, and the near-zero CIDEr is most likely a decoder-implementation artifact/bug rather than a fundamental property of the model.

CLAUDE.AI The core idea

You do not need to “port CaptioningHandler” or provide a “custom inference wrapper”. You can convert RSICD into the exact format Microsoft's COCO captioning pipeline already expects, then call the official `--eval` entry point. The generation loop (causal-masked, KV-cached, beam search) runs itself. The only RSICD-specific work is

- (1) producing a `dataset_coco.json` in Karpathy layout and
- (2) supplying an RSICD ground-truth json for the scorer.

Step 0 — Environment setup (Kaggle cell)

```
bash
# Clone the official repo
!git clone https://github.com/microsoft/unilm.git
%cd unilm/beit3

# Install deps (versions that work with the repo; pin to avoid the
# Python 3.12 breakages you already documented)
!pip install -q torchscale timm==0.4.12 sentencepiece pycocoevalcap pycocotools ftfy
!pip install -q "transformers==4.*" # XLNetRobertaTokenizer path; avoid 5.0 tokenizer breakage
```

A note on your six compatibility patches: keep them. If you run on Kaggle's current Python 3.12 / PyTorch image you'll re-hit the `torch._six`, `weights_only`, `tokenizer`, and `tensorboardX` issues. Apply the same patches you already wrote — they don't conflict with anything here. If you can use an older Kaggle image (PyTorch 2.1–2.3), several of them become unnecessary.

Step 1 — Lay out RSICD like the COCO captioning task expects

Microsoft's guide expects this directory shape (image folders + a Karpathy `dataset_coco.json`):

```
/kaggle/working/rsicd_data/
  images/          # all 10,921 RSICD JPEGs in one folder
  dataset_coco.json # Karpathy-format split file (you generate this)
```

RSICD images are normally one flat folder (`RSICD_images/`), so point at that. The real work is converting `dataset_rsicd.json` into Karpathy `dataset_coco.json` layout. The COCO/Karpathy schema the BEiT-3 loader reads is a top-level `{"images": [...]}` list where each entry has `split`, `filename`, `sentences` (each with a `raw` string and `tokenized` tokens), and an `imgid/cocoid`.

```
python
import json, os
```

```
RSICD_ANN = "/kaggle/input/your-rsicd/dataset_rsicd.json" # adjust path
OUT = "/kaggle/working/rsicd_data/dataset_coco.json"
```

```
rsicd = json.load(open(RSICD_ANN))

# RSICD's dataset_rsicd.json already follows the Karpathy-style schema
# (images -> sentences -> raw/tokens, with split labels train/val/test).
# We normalize field names to exactly what BEiT-3's CaptioningDataset reads.
images = []
for i, img in enumerate(rsicd["images"]):
    sents = []
    for s in img["sentences"]:
        raw = s["raw"].strip()
        toks = s.get("tokens") or raw.lower().replace(".", "").split()
        sents.append({"raw": raw, "tokens": toks, "sentid": s.get("sentid", len(sents))})
    images.append({
        "filename": img["filename"],
        "filepath": "images",          # subfolder under data_path
        "split": img["split"],          # "train" / "val" / "test"
        "sentences": sents,
        "imgid": i,
        "cocoid": i,                   # BEiT-3 loader uses an int image id
    })

os.makedirs("/kaggle/working/rsicd_data", exist_ok=True)
json.dump({"images": images}, open(OUT, "w"))
print("images:", len(images),
      "test:", sum(1 for x in images if x["split"] == "test"))
# expect ~10921 total, ~1093 test
```

Two things to verify after this runs: the `test` count should be **1093** (not 200), and every `filename` must exist inside `/kaggle/working/rsicd_data/images/`.

Step 2 — Build the BEiT-3 index files

This is the official call, straight from the captioning guide. It tokenizes captions with `beit3.spm` and writes the `.jsonl` index files the trainer/evaluator consume.

```
python
import sys
sys.path.append("/kaggle/working/unilm/beit3")

from datasets import CaptioningDataset
from transformers import XLMRobertaTokenizer

tokenizer = XLMRobertaTokenizer("/kaggle/input/your-beit3/beit3.spm") # adjust path

CaptioningDataset.make_coco_captioning_dataset_index(
    data_path="/kaggle/working/rsicd_data",
    tokenizer=tokenizer,
)
# writes coco_captioning.train.jsonl / .val.jsonl / .test.jsonl into data_path
```

If `make_coco_captioning_dataset_index` rejects your JSON, it's almost always a field-name mismatch from Step 1 — check that `split`, `filename`, `filepath`, and `sentences[] .raw` are all present.

Step 3 — The one RSICD-specific override: the ground-truth scoring file

This is the single trap that will otherwise silently break your CIDEr number. BEiT-3's `coco_caption_eval` (in `utils.py`) by default downloads a **COCO** GT json from Microsoft's storage and scores against it. On RSICD that would score your RSICD predictions against COCO references — garbage. You must supply an RSICD GT file in COCO-eval format and point the evaluator at it.

```
python
# Build a COCO-eval-format GT json for the RSICD test split
import json

rsicd = json.load(open("/kaggle/working/rsicd_data/dataset_coco.json"))
ann, imgs, aid = [], [], 0
for img in rsicd["images"]:
    if img["split"] != "test":
        continue
    imgs.append({"id": img["cocoid"]})
    for s in img["sentences"]:
        ann.append({"image_id": img["cocoid"], "id": aid, "caption": s["raw"]})
        aid += 1

gt = {"annotations": ann, "images": imgs,
      "type": "captions", "info": {}, "licenses": []}
json.dump(gt, open("/kaggle/working/rsicd_test_gt.json", "w"))
print("GT images:", len(imgs), "GT captions:", len(ann)) # ~1093 / ~5465
```

Then make the evaluator use it. The cleanest way is a tiny patch to `utils.py`'s `coco_caption_eval` so that for a `coco_captioning/test` split it loads your local file instead of downloading. Open `unilm/beit3/utils.py`, find the `filenames/urls` dict inside `coco_caption_eval`, and add a branch:

```
python
# inside coco_caption_eval(gt_dir, results_file, split), near the filename lookup:
if split == "test": # RSICD override
    annotation_file = "/kaggle/working/rsicd_test_gt.json"
else:
    # ...original COCO download logic...
    annotation_file = os.path.join(gt_dir, filenames[split])
```

If you'd rather not edit the file, an equivalent move is to pre-place `rsicd_test_gt.json` at the exact path/name the function expects (`coco_karpathy_test_gt.json` in `gt_dir`) so the existing code picks it up without downloading. Either works; the patch is more transparent for your reproducibility appendix.

Step 4 — Run the official eval

This is the command from Microsoft's captioning guide, adapted to one GPU and your RSICD data. It loads your fine-tuned checkpoint, runs the real autoregressive beam-search generation, writes predictions, and scores them.

```
bash
%cd /kaggle/working/unilm/beit3

!python -m torch.distributed.launch --nproc_per_node=1 run_beit3_finetuning.py \
  --model beit3_base_patch16_480 \
  --input_size 480 \
```

```
--task coco_captioning \  
--batch_size 16 \  
--sentencepiece_model /kaggle/input/your-beit3/beit3.spm \  
--finetune /kaggle/working/your_rsidc_finetuned_checkpoint.pth \  
--data_path /kaggle/working/rsicd_data \  
--output_dir /kaggle/working/beit3_eval_out \  
--eval \  
--dist_eval
```

Key flags, and why each matters:

- `--finetune` points at **your RSICD-fine-tuned checkpoint** (the one from your 3-epoch run), not the COCO checkpoint — that's what makes this a fair test of your trained model.
- `--eval` triggers the generation + scoring path instead of training.
- `--batch_size 16` is safe on a single T4; drop to 8 if you OOM at 480px.
- Beam size is set internally (the captioning task default is beam 3, matching your thesis config) — you don't pass it on the command line.

What you should see

The eval prints BLEU-1..4, METEOR, ROUGE-L, CIDEr, and SPICE for the 1,093-image RSICD test split, computed by `pycocoevalcap`. If the decoder was the problem — which the `torch.tril + incremental_state` evidence strongly suggests — your CIDEr should jump from 0.91 into a normal fine-tuned range (the literature for fine-tuned transformers on RSICD sits around 100–280 on the $\times 100$ scale; even a modest 3-epoch base-model run should land in the tens-to-low-hundreds, not below 1).

failure modes to watch

If captions come out coherent but CIDEr is still oddly low (say, single digits), suspect the GT file from Step 3 — confirm it has ~5465 captions across 1093 images and that `image_ids` match your prediction file's ids. If captions are still incoherent word-salad, then the eval path isn't actually being used (you've fallen back into a custom generation branch somewhere) — check that you're calling the unmodified `run_beit3_finetuning.py`, not a notebook reimplementation, and that `only_infer`/the captioning handler is being invoked.